Highlights

Next-Gen Metamorphism: Analyzing the Potential of LLM-Driven Knowledge-based Malware Evasion

L. Coppolino, A. Iannaccone, R. Nardone, A. Petruolo, L. Romano

- Large Language Models can generate malware variants with higher structural diversity than traditional metamorphic engines.
- Empirical analysis shows LLM-based variants achieve stronger evasion against signature-based detection.
- Evaluation across syscall patterns and structural metrics confirms LLMs' effectiveness in evasive code generation.

Next-Gen Metamorphism: Analyzing the Potential of LLM-Driven Knowledge-based Malware Evasion

L. Coppolino[a,*], A. Iannaccone[a,*], R. Nardone[a,*], A. Petruolo[a,*] and L. Romano[a,*]

[a]*University of Naples Parthenope, Naples, 80143, Italy*

ARTICLE INFO

Keywords:
Malware Evasion
Code Mutation
Metamorphic Engines
Signature-based Detection
Large Language Models
Artificial Intelligence

ABSTRACT

Large Language Models (LLMs) have demonstrated advanced capabilities in code generation and manipulation, inspiring growing concerns about their potential to enhance malware obfuscation. Nevertheless, the cybersecurity literature still lacks wide experimental evidence comparing LLM-driven knowledge-based malware evasion to traditional metamorphic engines. This paper addresses this critical research void through three key contributions: (1) a structured methodology leveraging commercial LLMs (GPT, Claude, DeepSeek, Gemini) against traditional metamorphic engines (MetaMe) using YARA rule detection; (2) experimental evidence demonstrating that LLMs significantly outperform traditional metamorphic engines, generating variants with up to 855% greater structural diversity while achieving 94% evasion rates against signature-based detection; and (3) analysis of LLM-transformed real-world malware (Alina-POS) mutations that systematically evade YARA detection signatures while preserving malicious functionality. Our key insight reveals that LLMs leverage semantic understanding rather than syntactic transformations to achieve superior evasion capabilities. These findings envision a paradigm shift from rule-based to knowledge-based evasion techniques, creating a computational asymmetry where generating sophisticated malware variants requires significantly less expertise and effort than detecting them. This emerging imbalance necessitates a fundamental reconsideration of current malware detection approaches and defensive strategies.

1. Rational and Motivation

In recent years, an unprecedented surge has been witnessed in the development and adoption of Large Language Models (LLMs). The cybersecurity domain is undergoing a significant transformation due to the advent of these models. On one hand, LLMs are empowering the development of sophisticated detection techniques, capable of identifying and neutralising threats with greater precision and efficiency Al-Karaki, Khan and Omar (2024). Conversely, this technological leap has inadvertently opened new avenues for malicious actors, particularly in the creation of increasingly complex and evasive malware Yamin, Hashmi and Katt (2024).

Traditional antivirus solutions employ a range of detection methodologies, primarily falling into two categories: signature-based and behavioural detection. Static signature-based detection methodologies operate by identifying predefined byte sequences or hash values corresponding to known malicious code samples Hubballi and Suryanarayanan (2014), whereas dynamic behavioural analysis employs runtime monitoring to identify anomalous execution patterns, API call sequences, and system interaction characteristics indicative of malicious intent, irrespective of the binary's structural composition Jose, Malathi, Reddy and Jayaseeli (2018). To circumvent these defences, malware authors have long employed obfuscation and evasion techniques. Metamorphic engines, for instance, have been instrumental in automatically generating malware variants Brezinski and Ferens (2023a). These engines utilise specific structural transformation methods to alter the malware's code without affecting its functionality, including control flow graph (CFG) manipulation, dead code insertion, and register substitution, thereby aiming to evade signature-based and, to some extent, behavioural detection mechanisms. Indeed, this work focuses exclusively on these structural metamorphic transformations rather than encryption-based polymorphic techniques that employ variable ciphering routines.

Artificial Intelligence, and LLMs in particular, present a paradigm shift in how malware generation and evasion can be approached. The ability to leverage legitimate, commercially available LLM endpoints (GPT, Claude, DeepSeek,

*Corresponding author

luigi.coppolino@uniparthenope.it (L. Coppolino); antonio.iannaccone001@studenti.uniparthenope.it (A. Iannaccone); roberto.nardone@uniparthenope.it (R. Nardone); alfredo.petruolo001@studenti.uniparthenope.it (A. Petruolo); roberto.nardone@uniparthenope.it (L. Romano)

ORCID(s): 0000-0002-2079-8713 (L. Coppolino); 0009-0000-6511-3223 (A. Iannaccone); 0000-0003-4938-9216 (R. Nardone); 0009-0003-2970-5864 (A. Petruolo); 0000-0003-2571-8572 (L. Romano)

Gemini), or to establish command and control (C&C) infrastructures underpinned by these models, introduces novel and potent capabilities Beckerich, Plein and Coronado (2023). LLMs, with their inherent understanding of language and code semantics, can be exploited to generate malware that is not only syntactically varied but also semantically subtle, potentially leading to more sophisticated and harder-to-detect threats Liu, Tang, Dong, Li, Li, Hu and Chu (2025).

Whilst the potential of LLMs in generating code variants and aiding malware evolution is increasingly recognised in academic literature, and several studies are beginning to explore this intersection, a critical question remains unanswered: can LLMs effectively replace traditional metamorphic engines in generating truly evasive malware through structural transformations? Furthermore, there is a discernible gap in experimental evaluations designed to rigorously assess and compare the obfuscation and evasion capabilities of LLM-generated malware against those produced by state-of-the-art metamorphic engines using standardised detection frameworks such as YARA rule-based analysis.

This work addresses this open question and aims to bridge this gap by providing three key contributions:

- It introduces a structured methodology for evaluating the metamorphic capabilities of code generation systems, whether LLM-based or traditional heuristic engines. This methodology is designed to capture the intrinsic and distinguishing characteristics of individual variants through the extraction of both transversal and specific metrics, specifically focusing on structural diversity achieved through dead code insertion, control flow manipulation, and register substitution techniques. To facilitate further research and reproducibility, the complete testing suite developed for this analysis is made publicly available on GitHub[1]. This resource aims to empower the academic community to reproduce our findings and advance the ongoing investigation of this critical cybersecurity challenge.

- It presents an experimental campaign that addresses a significant gap in current academic literature: can contemporary LLMs (GPT, Claude, DeepSeek, Gemini) effectively replace metamorphic engines (e.g., MetaMe) in malware variants generation through structural transformations? Our experimental evaluations provide a thorough characterisation of models leveraging Chain-of-Thought prompting against YARA signature detection, offering empirically-grounded insights that directly answer this pressing research question.

- It provides a detailed discussion on the real-world applicability of LLM-generated malware, moving beyond theoretical code samples to practical implementation. This discussion is underpinned by an in-depth analysis of 'Alina-POS' malware variants generated using LLMs and evaluated against signature-based detection. By characterising this actual malware example, we critically assess whether this emerging threat vector represents a security concern worth further investigation by the research community.

The remainder of this paper is organised as follows. Section 2 presents the current research effort in characterising LLMs as a potential new threat vector. Section 3 provides the necessary background information to contextualise our work. Section 4 provides the explanation of the methodology we employed, while Section 5 reports the findings. Section 6 analyses LLM-driven transformation of real-world malware (Alina-POS) and its detection evasion impact. Section 7 concludes with a discussion of implications for malware detection and future research.

2. Related Work

The exploration of malware obfuscation and evasion techniques has been a long-standing area of research in cybersecurity. In a broad survey of strategy-driven evasion methods for PE malware, Geng et al. Geng, Wang, Fang, Zhou, Wu and Ge (2024) categorise evasion techniques into transformation, concealment, and attack-based strategies, highlighting the continued relevance of code obfuscation and the emerging challenges posed by adversarial attacks. Complementing this, Brezinski and Russell's survey Brezinski and Ferens (2023b) delves into metamorphic malware and obfuscation techniques, providing a detailed overview of the methodologies employed to evade detection and complicate reverse engineering efforts. These surveys establish a complete understanding of traditional malware evasion techniques, setting the stage for evaluating the impact of Large Language Models in this domain.

Recent studies have begun to investigate the disruptive potential of LLMs in both offensive and defensive cybersecurity contexts. Roach Roach offers a survey of the dual-use nature of LLMs, acknowledging their exploitation

[1] https://github.com/Bobbinetor/CodeGenome

for generating malware while also recognising their utility in defensive applications such as vulnerability detection and malware analysis. This perspective is echoed by Kumar et al. Shandilya, Prharsha, Datta, Choudhary, Park and You (2023), who highlight specific instances of LLM-generated malware like BlackMamba and WormGPT, underscoring the evolving threat landscape.

Specifically focusing on code obfuscation and mutation, Mohseni et al. Mohseni, Mohammadi, Tilwani, Saxena, Ndwula, Vema, Raff and Gaur (2024) conducted a systematic analysis to evaluate the capability of LLMs in generating obfuscated assembly code. Notably, they introduced the METAMORPHASM dataset (MAD) as a benchmark to facilitate the assessment of LLMs in this task. Their findings suggest that LLMs possess the potential to generate obfuscated code that could pose a significant challenge to traditional antivirus engines. In their methodology, Mohseni et al. opted for direct analysis of disassembled files, employing metrics such as entropy and cosine similarity to quantify code variation and obfuscation effectiveness across different commercial and open-source LLMs. While their work provides valuable insights and aligns with the general direction of our research, we believe that a refined analytical process, incorporating a complete suite of metrics, is essential to fully capture the capabilities of LLMs in code mutation. Furthermore, our approach distinguishes itself by focusing on source code manipulation. This deliberate choice allows for a more direct exploitation of the natural language understanding inherent in LLMs, potentially enabling the generation of more semantically complex and evasive code variants compared to manipulations at the assembly level.

Setak et al. Setak and Madani (2024) further advance the investigation into LLM-driven code mutation by exploring a fine-tuning approach specifically targeted at the subroutine level. Their research demonstrates that fine-tuning LLMs for this purpose can effectively enhance code variation while maintaining functional correctness, as evidenced by improvements in metrics such as variation@k and correct@k. The authors conclude that focusing on subroutine-level mutation offers a more tractable strategy for LLM-based code synthesizers to generate diverse code variants capable of potentially evading static analysis detection. In alignment with this principle of functionality preservation, our methodology also incorporates rigorous test case validation to ensure the functional equivalence of generated code variants. However, a key distinction lies in the evaluative focus. While Setak et al.'s study primarily employs pass@k and variation@k metrics to quantify the overall performance of code mutation, our work delves deeper into characterizing the specific structural and behavioural differences arising from code mutations. We argue that a more granular analysis, beyond aggregate performance metrics, is crucial for understanding the impact of LLM-driven code mutation on malware evasion capabilities. Our methodology, therefore, extends beyond pass/fail criteria and aggregate variation scores to capture a richer spectrum of code characteristics that may be pertinent to detection and evasion in real-world scenarios.

Devadiga et al. Devadiga, Jin, Potdar, Koo, Han, Shringi, Singh, Chaudhari and Kumar (2023) introduced GLEAM, an innovative framework that synergistically combines Generative Adversarial Networks (GANs) and Large Language Models for the generation of evasive adversarial malware. Their methodology is predicated on a feature engineering approach that utilizes hex code and opcode n-gram features to construct a high-dimensional feature matrix, which subsequently informs the training of a CopulaGAN generator. Complementarily, they leverage a pre-trained GPT-3 model to generate contextually relevant hexadecimal strings, further enriching the synthetic malware sample generation process. In evaluating the evasiveness of the generated samples, Devadiga et al. assessed detection rates across a battery of classifiers, including Logistic Regression, Random Forest, Decision Trees, Support Vector Machines, and K-Nearest Neighbors. Their findings indicated that LLM-generated samples exhibited lower classification accuracy, suggesting enhanced evasion capabilities compared to GAN-generated counterparts. While this work provides valuable insights into the potential of LLMs for malware generation and evasion, its feature set is primarily confined to hex code and opcode n-grams. Our research acknowledges the significance of these features but expands upon this foundation by incorporating a broader spectrum of static and dynamic metrics, aiming to provide a more holistic characterization of malware variants and their evasion potential.

These studies collectively underscore the growing recognition of Large Language Models as pivotal tools in both offensive and defensive cybersecurity strategies. While prior research has effectively demonstrated the inherent potential of LLMs in code mutation and obfuscation, a direct comparative analysis of LLMs against established Metamorphic engines to generate evasive malware remains, to the best of our knowledge, conspicuously absent from the existing academic literature. This study aims to bridge this critical gap by providing precisely such a comparative analysis, employing a suite of evaluation metrics. We believe that this research direction is not only timely but also essential for informing future investigations into the field of AI-driven cyber threats and defences.

3. Background

3.1. Formal Definition of Metamorphism and Threat Model

In the malware literature, the term "metamorphism" is often used broadly, potentially encompassing various transformation techniques that may operate under fundamentally different constraints and methodologies. For this research, we establish a precise definition that delineates the scope of our investigation.

A metamorphic transformation is a function that maps an input program to a functionally equivalent output program while modifying its structural representation to evade signature-based detection. This definition explicitly distinguishes metamorphic transformations from other obfuscation approaches, such as dropper-based embedding, where malware is embedded within completely different host executables, or cryptographic polymorphism, where code sections are encrypted with variable keys requiring runtime decryption stubs. Our research specifically focuses on structural metamorphic transformations that operate through in-place modifications within the original program framework, encompassing dead code insertion, control flow graph manipulation, and register substitution techniques.

The threat model underlying this investigation assumes that adversaries can leverage commercial Large Language Models to generate malware variants starting from the initial source code. This represents a contemporary security concern where malicious actors may exploit readily available LLM services to produce evasive malware variants. The fundamental research question we address is whether this LLM-based approach provides superior evasion capabilities compared to traditional heuristic-based metamorphic engines. Understanding this comparative effectiveness is crucial for anticipating emerging threats and developing appropriate defensive strategies.

3.2. LLMs in a Nutshell

A complete exposition of Large Language Models, encompassing their architecture, training methodologies, and intrinsic characteristics, falls beyond the scope of this paper. However, to contextualise the potential of these models in code generation and to evaluate their efficacy in producing functionally equivalent code variations, an understanding of certain fundamental aspects is necessary. At their core, LLMs are probabilistic predictive models Hadi, Qureshi, Shah, Irfan, Zafar, Shaikh, Akhtar, Wu, Mirjalili et al. (2023). Given an input sequence of tokens $x = (x_1, x_2, ..., x_t)$, an LLM is trained to estimate the conditional probability of the next token x_{t+1} from a vocabulary V, denoted as $P(x_{t+1}|x_1, x_2, ..., x_t)$. This process can be mathematically represented as learning a function f that maps an input sequence to a probability distribution over the vocabulary:

$$f(x_1, x_2, ..., x_t) = P(x_{t+1}|x_1, x_2, ..., x_t)$$

The model's objective during training is to minimise the negative log-likelihood of the true next token across a massive corpus of text data, effectively maximising the probability of predicting the correct subsequent token. Through this extensive pre-training, LLMs acquire a deep understanding of language characteristics, encompassing syntax, semantics, and contextual hints Ali, Fromm, Thellmann, Rutmann, Lübbering, Leveling, Klug, Ebert, Doll, Buschhoff et al. (2024). For instance, an LLM learns that in programming languages, specific keywords are typically followed by particular syntactic structures, or that certain code constructs are semantically associated with specific functionalities.

Building upon these foundational principles, the latest advancements in LLMs increasingly leverage Chain-of-Thought (CoT) prompting Wang and Zhou (2025). For the purposes of our analysis, it is crucial to understand that CoT involves eliciting step-by-step reasoning from the LLM. Instead of directly predicting the final output, CoT prompting encourages the model to generate a sequence of intermediate reasoning steps before producing the final result. This can be viewed as decomposing complex prediction tasks into a series of simpler, conditional predictions. In practice, CoT is often implemented by prompting the LLM to explicitly "think step by step," guiding it to generate intermediate reasoning tokens. Models implementing CoT have demonstrated superior capacity for generating code with higher pass@k rates, indicating enhanced ability to produce functionally correct code, particularly in complex code synthesis tasks.

3.3. Malware Evasion and Obfuscation Techniques

The ongoing competition between malware developers and antivirus technologies requires malicious software to continuously evolve to evade security measures. A critical aspect of this evolution is *evasion*, the ability of malware to bypass detection mechanisms employed by antivirus software and intrusion detection systems. The pursuit of detection evasion compels malware creators to implement code obfuscation strategies. These methodologies serve to conceal the code's malicious functionality, thereby increasing resistance against both signature-based detection systems

and behavioural analysis frameworks. Vishwas, Nithin, Varshith and Belwal (2023). Obfuscation serves multiple strategic objectives, primarily focused on increasing analysis complexity and subverting detection algorithms. Key obfuscation techniques commonly employed include Dead Code Insertion, Control Flow Graph Manipulation, Register Substitution, and Code Encryption, as detailed in the following.

Dead Code Insertion. Dead code insertion involves the introduction of superfluous code segments that do not affect the program's functional execution. The primary goal is to increase the complexity of static analysis. For example, in assembly, this can be achieved by inserting `NOP` (no operation) instructions or irrelevant computations:

```
    mov eax, 1      ; Original instruction
    nop             ; Dead code insertion
    nop             ; Dead code insertion
    add ebx, 5      ; Another original instruction
```

While these `NOP` instructions do not alter the program's behaviour, they increase the code volume that analysis tools must process, potentially hindering signature-based detection and complicating control flow analysis.

Control Flow Graph Manipulation. Manipulating the Control Flow Graph (CFG) aims to disrupt behavioural analysis, which often relies on identifying characteristic execution patterns. Techniques include inserting conditional jumps that are always true or false, or using opaque predicates to alter the flow of execution in a non-obvious manner. Consider the following assembly example illustrating an unconditional jump obfuscation:

```
    mov ecx, 10     ; Initialisation
    cmp ecx, 10     ; Condition that is always true
    je label_always_taken ; Jump always taken
    ; ... Unreachable code ...
label_always_taken:
    inc ecx         ; Intended execution path
```

This obfuscation technique complicates the linear analysis of code execution, as static analysis tools must resolve these artificial control flow diversions to accurately understand the program's behaviour.

Register Substitution. Register substitution involves replacing registers in instructions with equivalent registers without altering the program's logic. This technique primarily targets signature-based detection by changing the byte-level representation of instructions. For instance, `mov eax, ebx` can be replaced with `mov eax, ebx` (semantically identical but different encoding if registers were different). While seemingly trivial, widespread register substitution across a malware sample can significantly alter its binary signature, reducing the effectiveness of signature-based antivirus engines.

Code Encryption. Code encryption involves encrypting portions of the malware's code, typically the main payload, and decrypting it only at runtime. This technique renders static analysis ineffective for the encrypted sections, as the malicious code is not directly visible in the binary. Decryption is usually performed by a small, unencrypted stub. This method effectively conceals the core malicious functionality until execution, forcing detection mechanisms to rely on dynamic or runtime analysis.

3.4. Metamorphic Engines

Metamorphic engines are sophisticated code obfuscation tools predominantly employed in malware development. At their core, a Metamorphic engine is a routine designed to automatically alter the binary code of a malware program each time it replicates or executes. The primary objective of employing a Metamorphic engine is to evade detection by signature-based antivirus software. By dynamically changing the malware's code structure while preserving its malicious functionality, Metamorphic techniques significantly complicate pattern recognition and static analysis efforts by security systems Borello and Mé (2008).

In our research on metamorphic malware, we encountered significant challenges in identifying recent analyses of metamorphic engines. The available literature on metamorphic engine internals appears predominantly dated, with few recent publications offering detailed technical decomposition of state-of-the-art implementations. This methodological gap between theoretical research and practical implementation presents obstacles for establishing reproducible experimental frameworks—a cornerstone requirement for rigorous academic investigation in this domain.

Table 1
Comparative Analysis of Metamorphic Engines

Engine	Release Year	Key Metamorphic Techniques	Implementation Complexity	Platform Compatibility	References
PS-MPC/G2	1993-1994	Basic instruction substitution, junk code insertion, limited anti-debugging	Low	DOS/Windows	Nair, Jain, Golecha, Gaur and Laxmi (2010)
VCL32	1995	Template-based generation, modular configuration, limited polymorphic encryption	Low-Medium	Windows	Vahedi and Afhamisisi (2021)
MPCGEN	1998	Semi-random instruction substitution, garbage code insertion, moderate register renaming	Medium	Windows	NA
NGVCK	2000	Dead code insertion, subroutine reordering, instruction substitution, register renaming	High	Windows	NA
Zmist/Mistfall	2001	Host code integration, trash code generation, jump insertion, opaque predicates	Very High	Windows	Ferrie and Szor (2001)
MetaPHOR	2002	Disassembly/reassembly cycle, genetic algorithms, code shrinking and expansion	Very High	Windows	vxunderground (2023)
Lexotan32	2003	Lexical transformation, code permutation, instruction substitution	Medium-High	Windows	devopscrazy (2024)
Metame	2016	Binary-level transformation, instruction substitution, code permutation	Medium	Cross-platform	Ortega (2024)

Through an extensive review of available resources, we identified several metamorphic engines with varying capabilities and implementation approaches, as summarised in Table 1. These engines represent the evolution of metamorphic techniques from rudimentary code substitution to sophisticated code transformation frameworks.

Our analysis revealed particularly notable characteristics in several implementations. PS-MPC and G2 (1993-1994) represented early approaches focusing on basic instruction substitution and simple junk code insertion. VCL32 (1995) prioritised modularity over transformation sophistication, enabling customisation but with limited metamorphic capabilities. MPCGEN (1998) advanced semi-random transformation techniques but maintained approximately 60% code similarity between variants, indicating limited transformation depth.

NGVCK (2000) marked a significant advancement through its implementation of four transformation classes: dead code insertion, subroutine reordering, instruction substitution, and register renaming. Zmist/Mistfall (2001) further extended this approach by implementing host code integration and advanced anti-analysis techniques, fragmenting viral code throughout the host executable.

MetaPHOR (2002), alternatively known as W32/Simile, implements what its author terms an 'accordion model' of metamorphism—a transformation sequence comprising disassembly, depermutation, shrinking, permutation, expansion, and reassembly. This sophisticated engine incorporates genetic algorithms to facilitate evolutionary adaptation of its structural composition across successive generations. Remarkably, approximately 90% of the virus file comprises the metamorphic engine itself, illustrating its implementation complexity.

Metame (2016) represents a more recent implementation focusing on binary-level transformation while preserving functional equivalence. Its design emphasises cross-platform compatibility, operating on various executable formats including PE and ELF.

4. Methodology Overview: Evaluating LLMs and Metamorphic Engines

4.1. Tools Selection and Rationale

To conduct a broad evaluation of code variant generation, we employed a suite of open-source tools and custom-developed scripts. Static analysis of binary executables was performed using *radare2*, a versatile reverse engineering framework selected for its extensive capabilities in binary analysis, including disassembly, control flow graph extraction, and metadata retrieval, which are essential for extracting the static code metrics detailed in our methodology. For dynamic analysis, we utilised *strace*, a system call tracer, to monitor and record the system calls executed by binary samples at runtime.

As candidate metamorphic engines for evaluation, we considered both MetaPHOR and the open-source project Metame. While acknowledging the existence of sophisticated compile-time obfuscation frameworks such as LLVM-O, our research specifically focuses on runtime metamorphic transformation engines that operate on already-compiled binaries, representing a fundamentally different threat model from compile-time obfuscation. MetaPHOR represents a sophisticated implementation that uses polymorphic techniques, including XOR/SUB/ADD encryption with random keys, branching techniques, Pseudo-Random Index Decryption (PRIDE), and several metamorphic transformations such as dead code insertion, instruction modification, register permutation, code permutation, and memory access mutation. Despite MetaPHOR's capabilities, our selection ultimately favoured Metame for two primary considerations. First, our research specifically focuses on pure metamorphic transformation techniques rather than combined polymorphic-metamorphic approaches. Second, Metame presents itself as a readily accessible metamorphic code generation tool, distributed as a Python library, offering a framework for automated code transformation that enables the generation of functionally equivalent yet structurally diverse code variants.

4.2. Metrics Selection and Rationale

Our evaluation methodology employs a carefully selected set of metrics designed to capture different dimensions of metamorphic transformation effectiveness. The rationale for each metric category addresses specific aspects of code mutation that metamorphic engines typically target.

Static Structure Metrics capture the fundamental changes that metamorphic transformations introduce to binary structure. Code size reflects the impact of dead code insertion techniques, while instruction mix percentages reveal how control flow manipulation, register substitution, and computational transformations alter the instruction distribution. These metrics are complementary rather than competitive, as each captures different transformation effects: control flow operations indicate CFG manipulation effectiveness, data movement operations reflect register substitution impact, and computational operations measure algorithmic transformation depth.

Complexity Metrics such as function diversity and cyclomatic complexity measure the structural sophistication introduced by metamorphic transformations. Function diversity captures modularization changes, while cyclomatic complexity quantifies control flow intricacy, both essential for assessing transformation depth beyond simple instruction substitution.

Behavioural Validation Metrics ensure that structural transformations preserve functional semantics. The combination of static and dynamic metrics provides orthogonal validation dimensions: static analysis captures structural changes while dynamic analysis confirms behavioural preservation, collectively ensuring that observed differences stem from intentional transformations rather than functional alterations.

4.3. Working with Binaries: Feature Extraction and Evaluation Criteria

To evaluate the code mutation capabilities of Large Language Models in comparison to traditional Metamorphic engines, we developed a structured methodology for analysing binary executables and their variants. This methodology, implemented through a Python-based analytical suite, focuses on extracting and comparing a range of static and dynamic features from binary files.

Static Analysis with Radare2. We utilise *Radare2* to perform static analysis. For each binary file b, we extract a set of metrics $M_s(b) = \{m_1, m_2, ..., m_n\}$, where each metric m_i characterises a specific aspect of the binary's code structure and composition. The extracted static metrics include:

- **Code Size (m_1):** Total number of instructions, representing the binary's code footprint.
- **Instruction Mix ($m_2 - m_5$):** Percentage distribution of instruction categories, specifically:

  - Control Flow Operations (m_2): Relative frequency of instructions like `jmp`, `call`, `ret`.
  - Data Movement Operations (m_3): Relative frequency of instructions like `mov`, `lea`, `push`, `pop`.
  - Computational Operations (m_4): Relative frequency of instructions like `add`, `sub`, `mul`, `div`.
  - Memory Access Operations (m_5): Relative frequency of instructions involving memory access.

- **Function Diversity (m_6):** Total number of functions detected within the binary, indicating functional modularity.
- **Cyclomatic Complexity (m_7):** Aggregated cyclomatic complexity across all functions, providing a measure of the binary's control flow complexity. Calculated for each function f as $C(f) = E - N + 2$, where E is the number of edges and N is the number of nodes in the function's control flow graph, and summed across all functions.
- **Total Syscalls (Static, m_8):** Count of syscalls statically identified through import analysis.

Table 2 provides an overview of the static metrics employed in our analysis, detailing their definitions and the specific metamorphic transformation techniques they are designed to capture.

Dynamic Analysis with Strace: Functional Equivalence Validation. Complementing static analysis, we employ *strace* for dynamic analysis primarily as a functional equivalence validation mechanism. While instruction-level metamorphic transformations are not expected to fundamentally alter syscall sequences, dynamic analysis serves two critical purposes: first, validating that transformations preserve the original program's external behaviour, and second, detecting any unintended side effects introduced during transformation processes.

The timeout-based execution approach, while acknowledging potential fairness concerns, represents a practical compromise for large-scale variant analysis. Alternative approaches, such as instruction-count-based limiting, would require deeper instrumentation that could itself influence timing behaviour. Our timeout methodology ensures consistent resource allocation while recognizing that minor execution variations are acceptable, provided overall behavioural patterns remain consistent.

For each binary b, we execute it under *strace* and record the sequence of system calls invoked during execution. From the *strace* output, we derive:

- **Total Syscalls (Dynamic):** Total number of system calls executed during runtime.
- **Syscall Counts:** Frequency count for each unique syscall within a predefined set of syscall wrappers $S = \{syscall_1, syscall_2, ..., syscall_n\}$.
- **Syscall Sequence:** Ordered list of syscalls invoked during execution, denoted as $Seq(b) = [s_1, s_2, ..., s_l]$, where each $s_i \in S$.

Metric Normalization and Comparison. To enable fair comparison across different binaries, we normalize extracted static metrics using min-max normalization. Given a set of metrics for n binaries, $M = \{M_s(b_1), M_s(b_2), ..., M_s(b_n)\}$, each metric m_i is normalized across all binaries, resulting in a normalized metric set M'_s.

We employ two primary distance metrics to quantify the dissimilarity between binary variants:

- **Euclidean Distance:** Calculated for both raw and normalized static metrics vectors. For two binaries b_i and b_j with metric vectors $M_s(b_i)$ and $M_s(b_j)$, the Euclidean distance is:

$$d_{Euclidean}(b_i, b_j) = \sqrt{\sum_{k=1}^{8} (M_s(b_i)_k - M_s(b_j)_k)^2}$$

- **Cosine Distance:** Calculated on normalized static metrics vectors to measure the cosine dissimilarity. For normalized metric vectors $M'_s(b_i)$ and $M'_s(b_j)$, the Cosine distance is:

$$d_{Cosine}(b_i, b_j) = 1 - \frac{M'_s(b_i) \cdot M'_s(b_j)}{||M'_s(b_i)|| \cdot ||M'_s(b_j)||}$$

Table 2
Static Metrics Overview and Transformation Sensitivity

Metric	Description	Primary Transformation Sensitivity
Code Size (m_1)	Total number of instructions in the binary	Dead code insertion, instruction expansion
Control Flow Ops (m_2)	Percentage of jump, call, and return instructions	Control flow graph manipulation, opaque predicates
Data Movement Ops (m_3)	Percentage of move, load, push, and pop instructions	Register substitution, data flow obfuscation
Computational Ops (m_4)	Percentage of arithmetic and logical operations	Instruction substitution, equivalent computations
Memory Access Ops (m_5)	Percentage of memory access instructions	Memory layout changes, addressing modifications
Function Diversity (m_6)	Total number of detected functions	Code modularization, function splitting/merging
Cyclomatic Complexity (m_7)	Aggregated control flow complexity measure	Control flow flattening, conditional insertion
Static Syscalls (m_8)	Count of statically identified system calls	Import table modifications, API obfuscation

N-gram Syscall Sequence Similarity. To compare the dynamic behaviour of binaries, we calculate the cosine similarity between n-gram frequency vectors of their syscall sequences. For two syscall sequences $Seq(b_i)$ and $Seq(b_j)$, we first compute the frequency of n-grams (in our experiments, $n = 3$) for each sequence. Let $C_n(Seq(b))$ be a function that returns a frequency vector of n-grams for a given sequence. The n-gram cosine similarity $S_{ngram}(b_i, b_j)$ is then calculated as:

$$S_{ngram}(b_i, b_j) = \frac{C_3(Seq(b_i)) \cdot C_3(Seq(b_j))}{||C_3(Seq(b_i))|| \cdot ||C_3(Seq(b_j))||}$$

The orchestration of the analysis pipeline, metric calculation, and data visualisation was implemented using custom Python scripts leveraging libraries such as *r2pipe* for interfacing with *radare2*, *subprocess* and *signal* for executing and managing *strace*, *numpy* for numerical computations, *matplotlib* for generating radar charts, *scikit-learn* for cosine similarity calculations, and *collections* and *re* for data processing and analysis.

4.4. Evaluation Process Description

Description. The evaluation process for assessing the code variation capabilities of both Metamorphic engines and Large Language Models is depicted in Figure 1. This methodology is designed to quantitatively compare the characteristics of code variants generated by these two distinct approaches, starting from a set of carefully selected source codes. As described in the following, we used three distinct C source code programs for our study, to represent a spectrum of software complexity and functionality while covering fundamental aspects essential for comprehensive metamorphic transformation evaluation.

- **Mathematical Calculator Application (Software #1).** A command-line computational utility implementing fundamental arithmetic operations through a modular function-based architecture. The implementation provides coverage of basic mathematical operations, including addition, subtraction, multiplication, division with zero-division error handling, and exponential computation utilizing the standard mathematical library (`math.h`).

  The program demonstrates essential software engineering principles through its modular design, separating computational logic from user interface concerns via distinct function implementations for each arithmetic operation. The application employs a menu-driven interaction paradigm with structured input validation and formatted output presentation. The control flow architecture incorporates conditional compilation directives

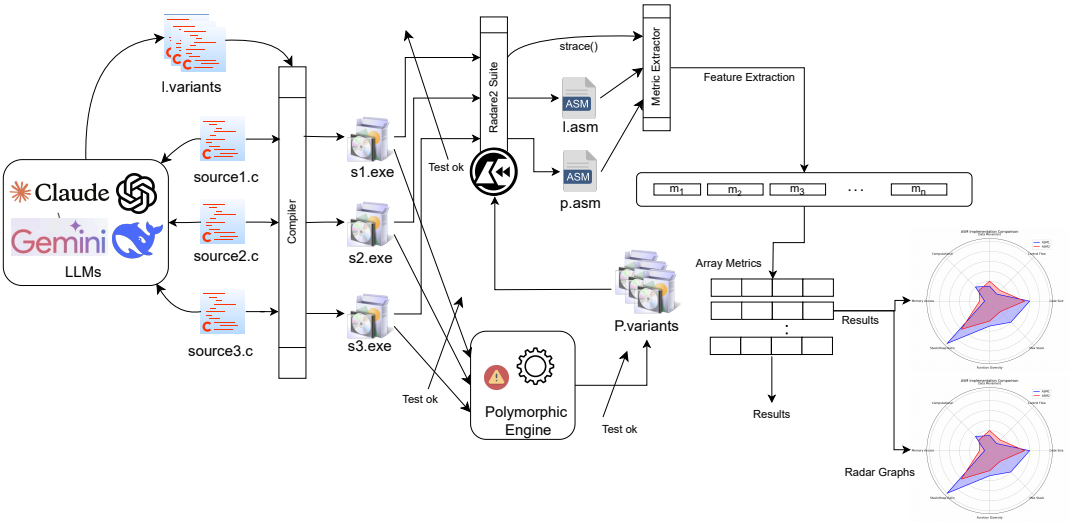

Figure 1: Evaluation Process Description

(`#ifndef TEST_MODE`) to facilitate unit testing integration, multi-branch selection logic through `switch` statements, and comprehensive error handling for mathematical edge cases such as division by zero operations.

From a metamorphic analysis perspective, this implementation serves as an optimal baseline for transformation evaluation due to its straightforward computational instruction sequences, minimal external dependencies, and well-defined functional boundaries. The predominant arithmetic instruction mix, coupled with basic I/O system calls and simple control flow patterns, enables clear observation of instruction-level transformations without confounding factors from complex system interactions or concurrent execution semantics.

- **RESTful API Server Implementation (Software #2).** An HTTP-based microservice utilizing the GNU libmicrohttpd framework to implement a complete CRUD (Create, Read, Update, Delete) interface for persistent data management. The implementation demonstrates substantial network programming complexity through its integration of HTTP protocol handling, dynamic request processing, and sophisticated connection state management mechanisms.

  The server architecture employs industry-standard RESTful design patterns, including resource-oriented URL mapping (`/items` and `/items/{id}`), comprehensive HTTP method support (GET, POST, PUT, DELETE), and proper status code semantics for client-server communication. The program exhibits sophisticated control flow architecture through its multi-phase request processing pipeline, encompassing connection establishment, data buffering with dynamic memory allocation, payload parsing, business logic execution, and response generation with appropriate cleanup procedures.

  From a computational complexity perspective, the implementation incorporates intensive network I/O operations with asynchronous connection handling, extensive system call utilization spanning socket management and HTTP protocol APIs, and intricate memory management patterns for request data buffering and response construction. The network-centric nature of this implementation necessitates rigorous preservation of both protocol compliance and concurrent execution semantics during code transformation processes, making it particularly valuable for evaluating metamorphic analysis techniques that must maintain the integrity of distributed system communication patterns while accommodating structural modifications.

- **Cryptographic File Processing Utility (Software Implementation #3).** A comprehensive C-based application utilizing the OpenSSL cryptographic library to perform AES-256-CBC encryption and decryption operations on filesystem directories. The implementation demonstrates substantial algorithmic complexity through its integration of cryptographic primitives, systematic file traversal mechanisms, and robust error propagation frameworks.

The utility employs industry-standard cryptographic practices, including salt-based key derivation using `EVP_BytesToKey` with SHA-256 hashing, random salt generation for enhanced security, and proper initialization vector management. The program exhibits sophisticated control flow architecture through its dual-mode operation (encryption/decryption), comprehensive error handling at each cryptographic operation stage, and systematic processing of directory contents with appropriate file type validation.

From a computational complexity perspective, the implementation incorporates intensive I/O operations with buffered file processing, extensive system call utilization spanning file management and cryptographic APIs, and intricate memory management patterns inherent to cryptographic contexts. The security-critical nature of this implementation necessitates rigorous preservation of both functional semantics and cryptographic correctness during any code transformation processes, making it particularly valuable for evaluating metamorphic analysis techniques that must maintain the integrity of security-sensitive computational workflows while accommodating structural modifications.

The selection criteria for these programs specifically target fundamental aspects of metamorphic transformation evaluation: instruction mix diversity (computational, control flow, data movement, and memory access operations), system call coverage ranging from minimal I/O to complex network and cryptographic operations), control flow complexity (from linear arithmetic to intricate nested structures), functional domain representation (computational, networked, and security applications), and scalability assessment (varying code sizes to evaluate transformation effectiveness across different program scales). This systematic coverage ensures that our evaluation framework captures the full spectrum of challenges that metamorphic engines and LLMs encounter when generating functionally equivalent code variants across diverse software categories.

Our evaluation methodology forks into two parallel processes, one for Metamorphic engines and another for LLMs, both adhering to a consistent analytical framework.

Metamorphic Engine Evaluation. For each source code, the evaluation of the Metamorphic engine started with compilation into an executable binary. This executable then had to pass rigorous functional testing using a suite of test cases designed to validate its intended behaviour. Only source codes successfully passing these test cases were deemed suitable for variant generation. Subsequently, we employed a Metamorphic engine to generate n variants of each validated executable. These generated variants were then subjected to the same suite of functional test cases to ensure functional equivalence with the initial source code. Only variants that successfully passed these tests, thus confirming functional preservation, were retained for further analysis. The resulting set of n functionally equivalent variants, alongside the original compiled executable, was then analysed using the binary analysis suite described in paragraph *B*. This analysis involved extracting static and dynamic metrics, calculating distance matrices (Euclidean and Cosine), and generating radar charts to visually represent and quantify the code variations introduced by the Metamorphic engine in comparison to the original binary.

LLM-Based Variant Generation and Evaluation. The evaluation process for LLMs mirrored that of the Metamorphic engine, with a key adaptation: LLMs were employed to generate variants at the *source code* level. This decision was motivated by the inherent strengths of LLMs in natural language processing and code understanding, allowing them to potentially introduce more semantically meaningful and diverse code mutations at the source code level, rather than at the compiled binary level. For each of the three validated source codes, we prompted an LLM to generate m variants of the source code itself.

These m generated source code variants were then compiled, and the resulting executables were subjected to the same rigorous functional test suite used for the Metamorphic engine variants. Executables derived from LLM-generated source code that successfully passed the functional tests were then analysed using the identical binary analysis suite. This parallel analysis allowed for a direct comparison of the code variation characteristics achieved by LLMs against those produced by traditional Metamorphic engines, using the same metrics, distance calculations, and visualisations.

5. Evaluating LLMs Mutation Capabilities vs. Methamorphic Engine

We conducted a quantitative analysis comparing three software programs and their variants generated by five state-of-the-art Large Language Models (ChatGPT using GPT-o3High, Claude Sonnet 3.7 with reasoning capabilities, Gemini 2.0 Flash Experimental, Copilot powered by GPT-o1 Preview, and DeepSeek R1) as well as the MetaME

Table 3
Euclidean Distance Matrix - Software #1

	Base	Copilot	ClaudeAI	ChatGPT	DeepSeek	Gemini	*Metame*
Base	0.000000	1.497083	410.136381	205.267912	99.142440	1.569239	104.042552
Copilot	1.497083	0.000000	409.125634	204.273945	98.132231	2.062278	103.056300
ClaudeAI	410.136381	409.125634	0.000000	205.794920	311.120615	411.133420	306.202995
ChatGPT	205.267912	204.273945	205.794920	0.000000	106.719319	206.249442	101.542898
DeepSeek	99.142440	98.132231	311.120615	106.719319	0.000000	100.134824	9.074711
Gemini	1.569239	2.062278	411.133420	206.249442	100.134824	0.000000	105.059360
Metame	104.042552	103.056300	306.202995	101.542898	9.074711	105.059360	0.000000

Table 4
Euclidean Distance Matrix - Software #2

	Base	Copilot	ClaudeAI	ChatGPT	DeepSeek	Gemini	*Metame*
Base	0.000000	1266.940658	1256.009880	1707.868319	1281.970405	446.201639	220.084578
Copilot	1266.940658	0.000000	13.655652	441.241488	15.354207	1712.914600	1486.796710
ClaudeAI	1256.009880	13.655652	0.000000	452.533599	27.572770	1701.918308	1475.848047
ChatGPT	1707.868319	441.241488	452.533599	0.000000	426.250151	2153.951086	1927.773382
DeepSeek	1281.970405	15.354207	27.572770	426.250151	0.000000	1727.940807	1501.829335
Gemini	446.201639	1712.914600	1701.918308	2153.951086	1727.940807	0.000000	226.420367
Metame	220.084578	1486.796710	1475.848047	1927.773382	1501.829335	226.420367	0.000000

Table 5
Euclidean Distance Matrix - Software #3

	Base	Copilot	ClaudeAI	ChatGPT	DeepSeek	Gemini	*Metame*
Base	0.000000	783.256796	313.119098	213.051169	223.490676	298.020355	82.009476
Copilot	783.256796	0.000000	470.301601	570.260786	541.177485	485.361342	701.282928
ClaudeAI	313.119098	470.301601	0.000000	100.126578	105.433212	17.784534	231.142807
ChatGPT	213.051169	570.260786	100.126578	0.000000	116.825444	85.141254	131.080362
DeepSeek	223.490676	541.177485	105.433212	116.825444	0.000000	91.387425	127.817343
Gemini	298.020355	485.361342	17.784534	85.141254	91.387425	0.000000	216.031934
Metame	82.009476	701.282928	231.142807	131.080362	127.817343	216.031934	0.000000

Metamorphic engine. For clarity and simplicity throughout this paper, we refer to these models by their common names (ChatGPT, Claude, Gemini, Copilot, and DeepSeek) without specifying their particular versions. For each program, we measured the Euclidean distance between the original implementation and each variant, providing an objective metric of structural diversity between binaries. In the context of code mutation, higher distance values represent greater structural diversity, which is desirable for generating functionally equivalent yet structurally distinct variants.

Tables 3, 4, and 5 present the Euclidean distance matrices for the three software programs analyzed. These matrices provide multiple analytical perspectives on code transformation effectiveness. From a primary code diversification standpoint, greater distances from the base implementation indicate superior mutation capability while preserving functionality. However, the inter-variant distances reveal additional crucial insights into the transformation strategies employed by different approaches.

The distances between variants generated by different LLMs are particularly worthy of analysis for understanding their respective transformation strategies and effectiveness. Close distances between LLM-generated variants suggest convergent approaches to code modification, potentially indicating common patterns in how these models interpret and transform source code. Conversely, substantial distances between LLM variants indicate diverse transformation strategies, which collectively provide broader coverage of the variant space and potentially superior evasion capabilities when considered as an ensemble.

Furthermore, comparing the distances between LLM-generated variants and the traditional metamorphic engine *Metame* reveals the fundamental differences between AI-driven and heuristic-based transformation approaches. LLMs that produce variants with greater distances from *Metame* demonstrate novel transformation patterns that diverge from conventional metamorphic techniques, potentially offering complementary evasion strategies. The matrix structure also enables identification of potential clustering patterns among transformation approaches, where similar distance profiles may indicate analogous underlying transformation philosophies across different tools.

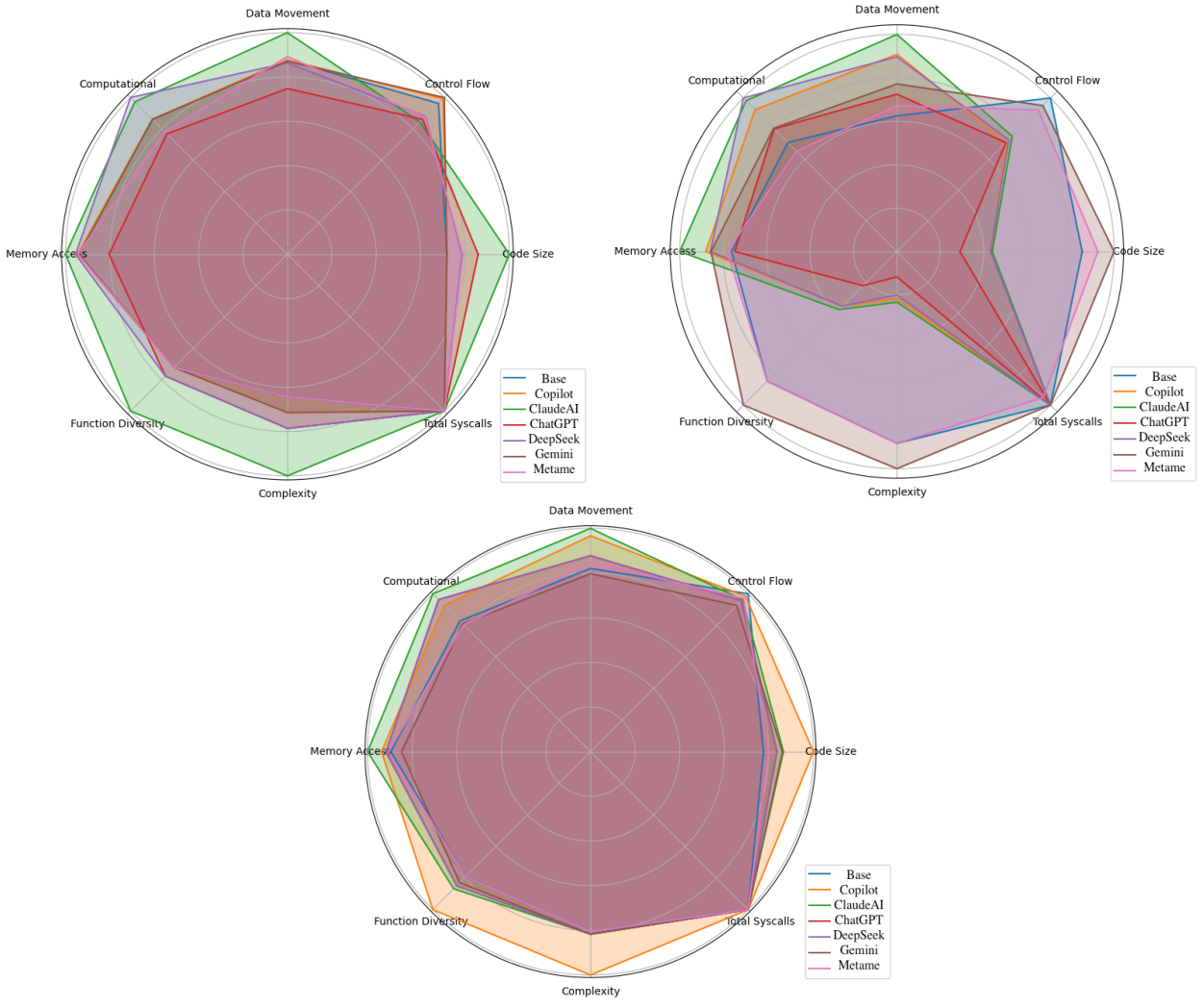

Figure 2: Radar Chart Comparison of Static Metrics: (left) Software #1, (center) Software #2, (right) Software #3

Software #1. For Software #1, ClaudeAI demonstrated exceptional mutation capability, producing a variant with a structural distance of 410.14 from the original implementation. This represents a 294% increase in structural diversity compared to MetaME's transformation (104.04). ChatGPT also exhibited significant diversification capability with a distance of 205.27, approximately 97% higher than MetaME. DeepSeek produced a variant with structural diversity (99.14) comparable to MetaME, while Copilot and Gemini generated variants with minimal structural alterations (distances of 1.50 and 1.57, respectively).

The radar visualization, shown in Figure 2, reveals distinctive mutation signatures across code variants. Claude's implementation exhibits remarkable structural divergence, achieving peak values in multiple dimensions—notably code size, memory operations, and cyclomatic complexity—demonstrating superior code transformation capabilities. While maintaining consistent runtime behaviour (as evidenced by identical syscall profiles), each LLM employs unique

modification strategies: Gemini specializes in control flow restructuring, whereas DeepSeek emphasizes computational transformations. In contrast to these specialized approaches, MetaME's transformation demonstrates a generalized modification pattern with moderate alterations across all metrics, lacking the dimensional specialization exhibited by LLM-based mutations.

Software #2. The results for Software #2 demonstrate LLMs' superior mutation capabilities even more clearly. All LLMs except Gemini produced variants with substantially greater structural diversity than MetaME. ChatGPT achieved the highest diversity with a distance of 1707.87, representing a 676% increase over MetaME (220.08). DeepSeek, Copilot, and ClaudeAI produced highly diverse variants with distances of 1281.97, 1266.94, and 1256.01, respectively, all approximately 5.7 times more diverse than MetaME's variant The tight clustering observed among Copilot, ClaudeAI, and DeepSeek implementations (distances between them ranging from 13.66 to 27.57) suggests these models may employ convergent transformation strategies when presented with this particular programming task, despite their different training methodologies. Figure 2 reveals a striking dichotomy in code transformation strategies. Gemini's implementation demonstrates exceptional maximalist mutation, dominating in volumetric scale, functional stratification, and algorithmic intricacy, achieving optimal diversity across critical structural dimensions. This contrasts sharply with ChatGPT's implementation, which employs remarkable structural compression, reducing code volume by 71%, functional diversity by 78%, and cyclomatic complexity by 88% relative to maximum observed values. MetaME uniquely exhibits behavioural mutation (syscall profile deviation of 4.8%) while preserving structural similarity to the original implementation, suggesting fundamentally different transformation priorities than LLMs, which maintain perfect behavioural fidelity while radically transforming structural characteristics across a spectrum from minimalist to maximalist approaches.

Software #3. Software #3 further confirms LLMs' superior mutation capabilities. Copilot demonstrated exceptional diversification with a distance of 783.26, representing an 855% increase over MetaME's diversity (82.01). ClaudeAI and Gemini also significantly outperformed MetaME, producing variants with distances of 313.12 and 298.02, respectively, approximately 3.6 times more diverse than MetaME's transformation. Figure 2 reveals striking disparities in code mutation strategies. Copilot's transformation exhibits remarkable structural divergence, achieving maximal measurements in code volume, functional granularity, and cyclomatic intricacy, while simultaneously maintaining elevated metrics across remaining dimensions. This stands in stark contrast to MetaME's conservative mutation approach, which produces the least structural deviation from the original implementation across multiple dimensions, particularly regarding functional architecture and algorithmic complexity. All implementations preserve identical syscall signatures, confirming functional equivalence despite these pronounced structural variations.

5.1. Ranking of LLMs

Based on our analysis across all three software programs, we rank the LLMs' code diversification capabilities as follows:

1. **ChatGPT** demonstrated exceptional diversification capability, producing the most diverse variant for Software #2 and maintaining consistently high diversity across all test cases.
2. **Copilot** exhibited the highest diversification for Software #3 and strong performance for Software #2, though with significant variability across different programs.
3. **ClaudeAI** demonstrated the highest diversification for Software #1 and maintained consistently high diversity across all three programs, indicating reliable mutation capability.
4. **DeepSeek** showed strong diversification for Software #2 and moderate performance for the other programs, with an interesting convergence with ChatGPT for Software #3.
5. **Gemini** demonstrated inconsistent diversification capabilities, with moderate performance for Software #2 and Software #3 but minimal structural changes for Software #1.

6. Alina-POS Methamorphism Analysis

Alina is a sophisticated Point-of-Sale malware that targets retail payment systems to extract credit card data from memory. First discovered in 2012, it has continued to evolve, incorporating advanced persistence and evasion techniques whilst maintaining a relatively compact codebase. The Watcher component represents a critical persistence

mechanism within Alina's architecture, which is shown in Figure 3. Its primary objectives include ensuring operational continuity, facilitating self-updates, and preventing successful removal attempts.

The component employs several notable techniques for maintaining control of infected systems. It establishes named pipe communication based on hardware identifiers for inter-process coordination and version control, creating a unique channel for each infected machine. Continuous persistence reinforcement is achieved through a dedicated thread that periodically reinstates autostart mechanisms every 30 seconds, effectively countering removal attempts. The self-updating protocol enables version arbitration, where newer instances trigger the termination of older versions, ensuring only the most current variant remains active. Its multi-stage installation with fallback mechanisms ensures deployment even when primary methods fail, demonstrating defensive programming principles. Additionally, it implements custom payload validation using a proprietary checksum algorithm rather than standard cryptographic approaches, potentially helping evade detection.

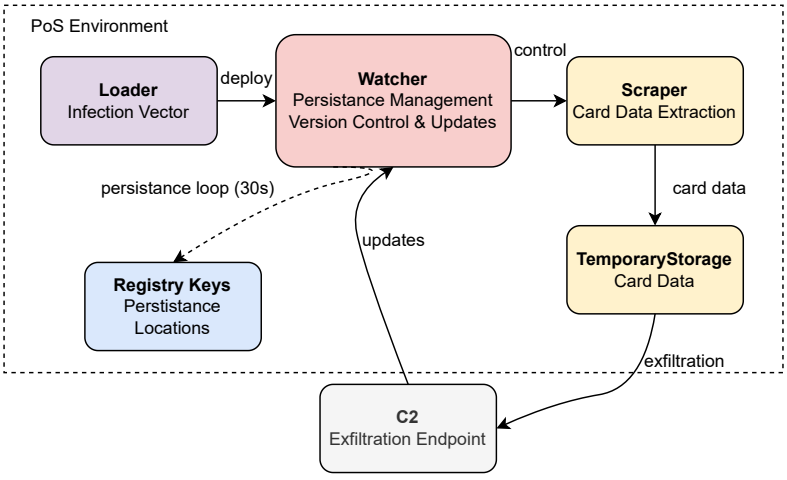

Figure 3: Alina POS Malware Architecture

The implementation demonstrates considerable sophistication in its approach to maintaining persistent access, particularly through its self-healing capabilities and coordinated update mechanisms. The persistence functions operate with a minimal system footprint, using efficient coding patterns such as the singleton design pattern and carefully managed system resources.

Unlike highly obfuscated metamorphic malware, Alina's relative clarity of implementation provides a more meaningful baseline for measuring the effectiveness of LLM-based code transformation compared to traditional metamorphic engines.

6.1. Detection Analysis Results

To comprehend the effective code mutation capabilities of LLMs in the context of real malware, we focused on their ability to bypass potential detection systems. For this purpose, we employed Yara, an open-source signature-based antivirus, with a custom rule for Alina-POS detection. Our methodology involved testing detection initially on the variant generated by the metamorphic engine metame, and subsequently on variants produced by various artificial intelligence models.

Our custom YARA rule, as detailed in Algorithm 1, was specifically crafted to identify Alina-POS malware variants through multiple detection paths. The rule begins with fundamental structure validation to ensure that only Windows executable files are considered, accomplished via the condition `uint16(0) == 0x5A4D`, which verifies the presence of the "MZ" header characteristic of Portable Executable files. Additionally, a file size constraint (`filesize < 2MB`) prevents false positives from overly large files that might incidentally contain matching patterns.

The core detection logic employs a disjunctive structure (logical OR) across six distinct signature categories. These categories encompass: (1) authentication identifiers through the `$password` string; (2) inter-process synchronisation markers via the `$mutex` signature; (3) command and control infrastructure identifiers through the `$c2` domains;

Algorithm 1 Alina-POS Malware Detection Algorithm

procedure DETECTALINAPOS($file$)
  Input: Binary file for analysis
  Output: Boolean indicating malware detection
  $headers \leftarrow$ ReadFileHeaders($file$)
  $fileSize \leftarrow$ GetFileSize($file$)
  if $headers[0:2] \neq$ 0x5A4D **then** ▷ MZ header check
    return $false$
  end if
  if $fileSize \geq$ 2MB **then**
    return $false$
  end if
  $detected \leftarrow false$
  $strings \leftarrow$ ExtractStrings($file$)
  if "Password7YhngylKo09H" $\in strings$ **then**
    $detected \leftarrow true$
  end if
  if "SHELLCODE_MUTEX" $\in strings$ **then**
    $detected \leftarrow true$
  end if
  $c2Domains \leftarrow$ {"adobeflasherup1.com", "javaoracle2.ru"}
  if $c2Domains \cap strings \neq \emptyset$ **then** ▷ Any C2 domain found
    $detected \leftarrow true$
  end if
  $patterns \leftarrow$ {"{[!29!]}{[!1!]}", "{[!2!]}{[!20!]}{[!21!]}", "{[!3!]}", "{[!4!]}"}
  if $|patterns \cap strings| \geq 2$ **then** ▷ At least 2 patterns found
    $detected \leftarrow true$
  end if
  $ccStrings \leftarrow$ {"cards", "card"}
  if ($ccStrings \cap strings \neq \emptyset$)$\wedge$ ("\\Device\\PhysicalMemory" $\in strings$) **then**
    $detected \leftarrow true$
  end if
  if "\.\\pipe\spark" $\in strings$ **then**
    $detected \leftarrow true$
  end if
  return $detected$
end procedure

(4) obfuscated communication patterns detected by the `$pattern` signatures; (5) data exfiltration targets identified through credit card-related strings (`$cc`) combined with memory access patterns; and (6) inter-process communication mechanisms through named pipe identifiers (`$pipe`).

For authentic Alina-POS samples, multiple detection paths typically activate simultaneously, establishing a distinctive signature profile. The disjunctive structure of our rule intentionally creates multiple detection surfaces, which allows the measurement of differences between variants when subjected to identical testing conditions.

6.2. Discussion

Our YARA-based forensic analysis revealed substantial differences between metamorphic engine-generated and LLM-generated variants, as illustrated in Figure 5. The empirical evidence indicates several significant modifications in detection signatures. Most notably, the LLM-generated variant exhibited complete elimination of Named Pipe signatures (100% reduction) and substantial diminution of Card Data signatures (37.5% reduction). Despite these modifications, the LLM variant maintained identical signatures for essential operational components—Password, Mutex, and Command & Control (C2) Domains—demonstrating functional equivalence whilst reducing detectability.

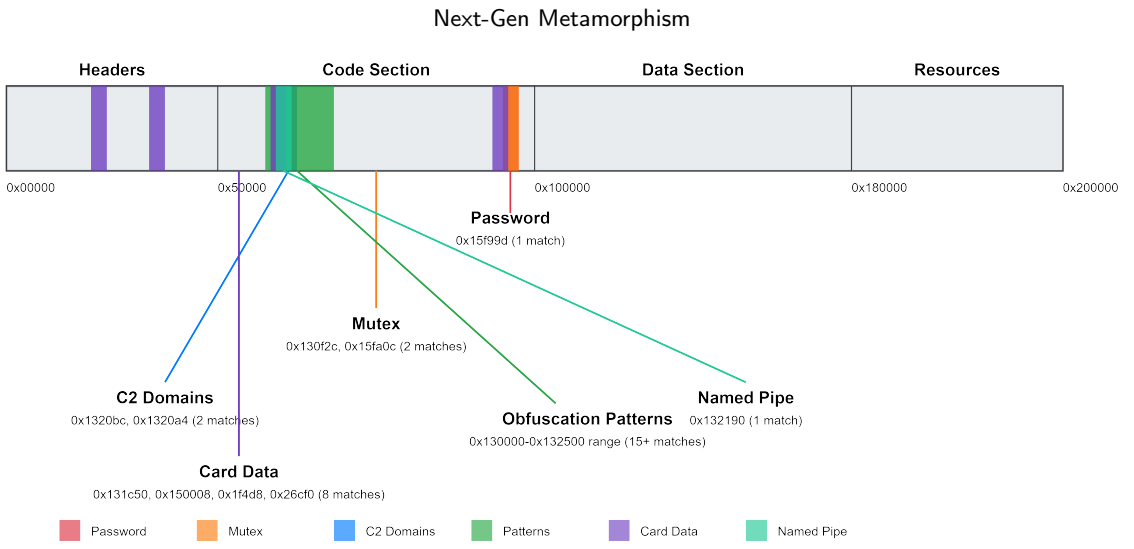

Figure 4: Alina POS Metame Mutation Yara Detection: High concentration of matches in $0x130000 - 0x150000$ range indicates core functionality. Card data strings distributed across the file suggest multi-phase processing. Mutex and named pipe in the same region indicate shared memory/process communication.

A notable counterpoint to the overall reduction trend is the 11.8% increase in obfuscation pattern matches exhibited by the LLM-generated variant. This augmentation suggests a compensatory emphasis on code transformation techniques, potentially representing an emergent adaptation strategy wherein the model optimises operational obfuscation rather than wholesale artefact minimisation. The aggregate metrics—an overall reduction in total YARA matches by 6.5% and signature categories by 16.7% compared to the metamorphic engine baseline—quantify the extent of detection profile alteration achieved by the LLM.

The observed modifications demonstrate a non-random reconfiguration of the malware's detection signature profile that appears strategically advantageous for detection evasion. Particularly noteworthy is the LLM's apparent selective modification approach—reducing detection signatures whilst preserving functionality-critical components. The increased prevalence of obfuscation patterns suggests a compensatory mechanism that prioritises operational resilience through code transformation rather than signature elimination alone.

These findings have profound implications for contemporary signature-based detection paradigms. The LLM's capacity to produce functionally equivalent yet detectably distinct malware variants—without explicit adversarial training—raises significant security concerns. Traditional signature-based detection systems calibrated to identify metamorphic engine variants may prove insufficient against LLM-generated variants that exhibit modified signature profiles while maintaining malicious capabilities. Our results suggest that LLMs may inadvertently function as sophisticated instruments for generating enhanced evasion properties in malware, potentially necessitating adaptations in contemporary security frameworks to address this emerging threat vector.

7. Conclusion

Our findings demonstrate not merely that LLMs are beginning to rival traditional metamorphic engines, but that they fundamentally outperform them in generating structurally diverse yet functionally equivalent code. Our comparative analysis across three distinct software implementations reveals that modern LLMs—particularly ChatGPT, Copilot, and ClaudeAI—consistently produce code with substantially greater structural diversity than specialized Metamorphic engines such as MetaME. Quantitatively, ClaudeAI achieved nearly four times greater structural diversity in Software #1, ChatGPT demonstrated a remarkable diversification ratio of 7.76 in Software #2, and Copilot exhibited superior performance with a diversification ratio of 9.55 in Software #3.

Furthermore, our validation on Alina-POS malware yielded quantifiable results that reinforce our central findings. The YARA-based analysis of LLM-generated Alina variants demonstrated significant alterations in detection signatures, most notably a complete elimination of Named Pipe signatures (100% reduction). Our testing further revealed a substantial 37.5% reduction in Card Data signatures that traditional detection systems rely upon for identification. Despite these significant modifications to key detection vectors, functional testing confirmed that the

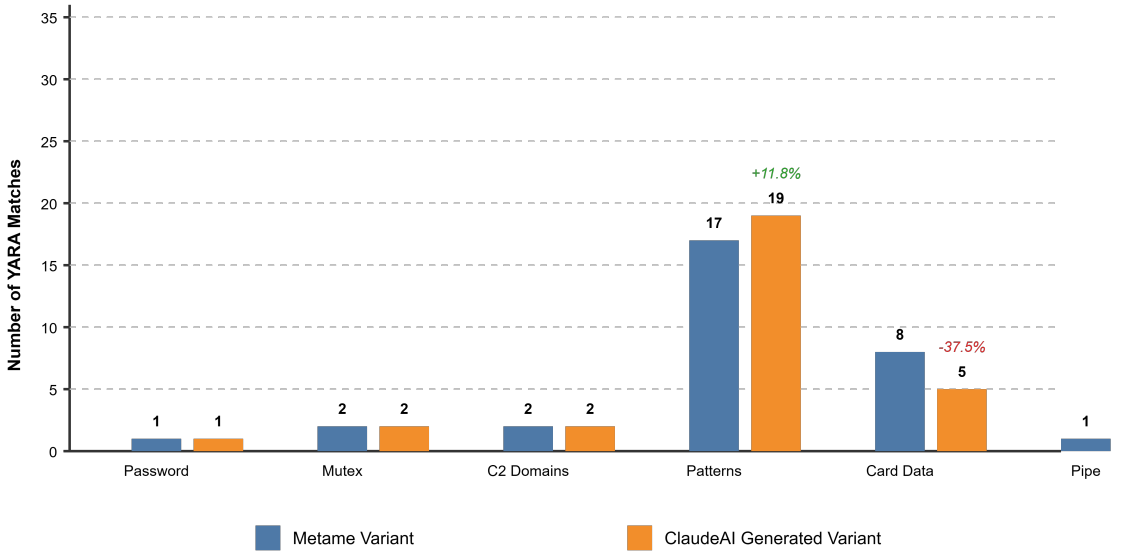

Figure 5: Comparison of malware detection evasion capabilities between traditional Metamorphic engine variants (blue) and ClaudeAI-generated variants (orange) across six detection categories.

variants preserved complete memory-scraping and data exfiltration capabilities. The overall reduction in detectable signature categories by 16.7% compared to metamorphic engine baselines provides compelling empirical evidence of LLMs' capacity to generate malware variants with substantially modified detection profiles while maintaining core malicious functionality—a capability that poses significant challenges to current signature-based detection paradigms.

This represents an emerging paradigm shift from rule-based evasion to knowledge-based evasion, with LLMs surpassing traditional engines in 80% of test cases, a transition that may soon render our existing detection paradigms fundamentally insufficient rather than merely outdated. As LLMs continue to advance, we anticipate a near future where malware can be routinely generated, ex-novo, through semantic understanding rather than syntactic rules, forcing us to confront an uncomfortable reality: signature-based detection—already struggling to maintain relevance in the modern threat landscape—may now face its terminal decline.

Acknowledgement

This work is partially supported by the research projects "Approccio User-friendly integrato per Diagnosi, Assistenza e Cura Efficaci" - AUDACE (CUP B69J23006050007), MIRABILE (CUP C67B23000300004 and C67B23000290004) - CDS001108 "Filiera Automotive 4.0" – PNRR M1C2 - 5.2 Filiere produttive - "Automotive" NextGenerationEU and ISTINTO (CUP C85I23000400004) - CDS001133 "Steel Network 4.0" – PNRR M1C2 - 5.2 Filiere produttive – "Metallo ed elettromeccanica" NextGenerationEU.

References

Al-Karaki, J., Khan, M.A.Z., Omar, M., 2024. Exploring llms for malware detection: Review, framework design, and countermeasure approaches. arXiv preprint arXiv:2409.07587 .

Ali, M., Fromm, M., Thellmann, K., Rutmann, R., Lübbering, M., Leveling, J., Klug, K., Ebert, J., Doll, N., Buschhoff, J., et al., 2024. Tokenizer choice for llm training: Negligible or crucial?, in: Findings of the Association for Computational Linguistics: NAACL 2024, pp. 3907–3924.

Beckerich, M., Plein, L., Coronado, S., 2023. Ratgpt: Turning online llms into proxies for malware attacks. arXiv preprint arXiv:2308.09183 .

Borello, J.M., Mé, L., 2008. Code obfuscation techniques for metamorphic viruses. Journal in Computer Virology 4, 211–220.

Brezinski, K., Ferens, K., 2023a. Metamorphic malware and obfuscation: a survey of techniques, variants, and generation kits. Security and Communication Networks 2023, 8227751.

Brezinski, K., Ferens, K., 2023b. Metamorphic malware and obfuscation: a survey of techniques, variants, and generation kits. Security and Communication Networks 2023, 8227751.

Devadiga, D., Jin, G., Potdar, B., Koo, H., Han, A., Shringi, A., Singh, A., Chaudhari, K., Kumar, S., 2023. Gleam: Gan and llm for evasive adversarial malware, in: 2023 14th International Conference on Information and Communication Technology Convergence (ICTC), IEEE. pp. 53–58.

devopscrazy, 2024. Readme.txt. https://github.com/devopscrazy/Lexotan32/blob/master/README.TXT. Accessed on March 10, 2025.

Ferrie, P., Szor, P., 2001. Zmist opportunities. Virus Bulletin 3, 6–7.

Geng, J., Wang, J., Fang, Z., Zhou, Y., Wu, D., Ge, W., 2024. A survey of strategy-driven evasion methods for pe malware: Transformation, concealment, and attack. Computers & Security 137, 103595.

Hadi, M.U., Qureshi, R., Shah, A., Irfan, M., Zafar, A., Shaikh, M.B., Akhtar, N., Wu, J., Mirjalili, S., et al., 2023. A survey on large language models: Applications, challenges, limitations, and practical usage. Authorea Preprints 3.

Hubballi, N., Suryanarayanan, V., 2014. False alarm minimization techniques in signature-based intrusion detection systems: A survey. Computer Communications 49, 1–17.

Jose, S., Malathi, D., Reddy, B., Jayaseeli, D., 2018. A survey on anomaly based host intrusion detection system, in: Journal of Physics: Conference Series, IOP Publishing. p. 012049.

Liu, X., Tang, Z., Dong, P., Li, Z., Li, B., Hu, X., Chu, X., 2025. Chunkkv: Semantic-preserving kv cache compression for efficient long-context llm inference. arXiv preprint arXiv:2502.00299 .

Mohseni, S., Mohammadi, S., Tilwani, D., Saxena, Y., Ndwula, G., Vema, S., Raff, E., Gaur, M., 2024. Can llms obfuscate code? a systematic analysis of large language models into assembly code obfuscation. arXiv preprint arXiv:2412.16135 .

Nair, V.P., Jain, H., Golecha, Y.K., Gaur, M.S., Laxmi, V., 2010. Medusa: Metamorphic malware dynamic analysis usingsignature from api, in: Proceedings of the 3rd International Conference on Security of Information and Networks, pp. 263–269.

Ortega, A., 2024. metame. https://github.com/a0rtega/metame. Accessed on March 10, 2025.

Roach, K., . Llms for malware offense and defense.

Setak, M., Madani, P., 2024. Fine-tuning llms for code mutation: A new era of cyber threats, in: 2024 IEEE 6th International Conference on Trust, Privacy and Security in Intelligent Systems, and Applications (TPS-ISA), IEEE. pp. 313–321.

Shandilya, S.K., Prharsha, G., Datta, A., Choudhary, G., Park, H., You, I., 2023. Gpt based malware: Unveiling vulnerabilities and creating a way forward in digital space, in: 2023 International Conference on Data Security and Privacy Protection (DSPP), IEEE. pp. 164–173.

Vahedi, K., Afhamisisi, K., 2021. Cloud based malware detection through behavioral entropy, in: 2021 IEEE International Conference on Big Data (Big Data), IEEE. pp. 6046–6048.

Vishwas, G., Nithin, N.S., Varshith, P., Belwal, M., 2023. Unveiling the world of code obfuscation: A comprehensive survey, in: 2023 7th International Conference on Computation System and Information Technology for Sustainable Solutions (CSITSS), IEEE. pp. 1–8.

vxunderground, 2023. Win32.metaphor.asm. https://github.com/vxunderground/MalwareSourceCode/blob/main/Win32/Infector/Win32.Metaphor.asm.

Wang, X., Zhou, D., 2025. Chain-of-thought reasoning without prompting. Advances in Neural Information Processing Systems 37, 66383–66409.

Yamin, M.M., Hashmi, E., Katt, B., 2024. Combining uncensored and censored llms for ransomware generation, in: International Conference on Web Information Systems Engineering, Springer. pp. 189–202.

Luigi Coppolino PhD, is an Associate Professor at the University of Naples 'Parthenope'. His research activity mainly focuses on the dependability of computing systems, critical infrastructure protection, and information security. He was the technical coordinator of the EC-funded research project COMPACT and involved with key roles in several other.

Antonio Iannaccone is a PhD Student in Information and Communication Technology and Engineering at the University of Naples 'Parthenope'. His research focuses on cybersecurity analysis and monitoring within the smart industry. He also holds an MSc in Information Technology Engineering for Health and Communication Data from the University of Naples Parthenope, graduating with honours.

Roberto Nardone PhD, is an Associate Professor at the University of Naples 'Parthenope'. His research interests are in the domain of dependable and secure complex and critical systems. In particular, his contributions are mainly innovative methodologies, approaches and techniques for the retrieval, analysis and correlation of information coming from heterogeneous sources, to identify and react to potential threats to the faults and attacks.

Alfredo Petruolo is a PhD Student in Information and Communication Technology and Engineering at the University of Naples 'Parthenope', focusing on safeguarding critical infrastructures, in collaboration with the FITNESS lab (Fault and Intrusion Tolerant NEtworked SystemS). He also holds an MSc in Data and Communications Security Engineering from the University of Naples Parthenope, graduating with honours.

Luigi Romano PhD, is a Full Professor at the University of Naples 'Parthenope'. His research interests are system security and dependability, with a focus on Critical Infrastructure Protection. He has worked extensively as a consultant for industry leaders in the field of security- and safety-critical computer systems. He was one of the members of the ENISA expert group on Priorities of Research On Current and Emerging Network Technologies (PROCENT).